

2주차 - C언어 기초1

비트와 바이트

비트 : 가장 작은 기본 정보 단위

바이트 : 연속적인 8개의 비트가 모인 정보 단위

프로그래밍 언어

프로그래밍 언어 : 컴퓨터와 의사교환을 하기 위한 언어. 사람이 읽기 쉬우나 컴퓨터는 해석할 수 없음

기계어 : 컴퓨터가 인식할 수 있는 언어. 컴퓨터는 인식할 수 있으나, 사람이 해석하기 매우 어려움

컴파일러 vs 인터프리터

C언어 : 속도가 빠름. 메모리 효율적으로 운용 가능. 컴파일러 언어

파이썬 : 이해, 습득하기 쉬움. 메모리 관리가 편함. 인터프리터 언어

C언어의 동작 과정

컴파일러 언어이기 때문에 모든 소스를 번역 후 실행파일을 생성. 소스코드 -> 실행파일 과정으로 진행
소스코드 -> 전처리소스 -> 어셈블리코드 -> 오브젝트파일 -> 실행파일

입출력

printf : 자료내용을 화면으로 출력

scanf : 키보드입력을 자료로 입력

#include <stdio.h> : standard IO 라이브러리 - 표준입력과 출력에 대한 함수

변수

데이터를 담는 그릇. 변할 수 있는 수

cf) 상수 : 1, "hello", 12.45 등 이미 값이 정해진 수

변수의 생성 : 자료형과 변수이름을 조합하여 선언

변수 이름 규칙

1. 대소문자 구분
2. 영문자, 숫자, 언더바(_)로 구성 - 숫자로 시작 불가
3. 공백 사용 불가, 예약어 사용 불가

자료형

컴퓨터는 2진수의 정보만을 인식

기본형, 유도형, 사용자 정의형.

정수형(short, int, long)

실수형(double float)

문자형

(char) 무치형(void)

자료형의 크기 1/2/4/8바이트... 크기가 클수록 많은 정보를 표현. but 사용하는 메모리의 양이 증가

시스템에 따라 자료형별 크기가 다를 수 있음. 목적과 시스템에 맞는 적절한 자료형을 결정하는 것이 중요.

자료형의 크기를 지정하지 않으면, 가져올 데이터의 시작위치부터 어디까지 값을 읽어와야 할지 모르게 됨!

signed/unsigned

n개의 비트로 표현가능한 범위는 2^n 개의 정보 표현 가능.

n바이트가 표현 가능한 범위는? 2^{8n} 개의 정보표현 가능

unsigned로 하면 양수만 표현 가능! 범위가 2배로 늘어남.

overflow : 표현 가능한 최대/최솟값을 넘었을 경우

2의 보수 : signed 자료형은 2의 보수로 음수를 표현함. 1의 보수(비트전환) 후 1 더하기!

	표현자료	크기	표현 범위 (signed)	표현 범위 (unsigned)
int	정수	4 bytes	$-2,147,483,648 \sim 2,147,483,647$ ($-2^{31} \sim 2^{31} - 1$)	$0 \sim 4,294,967,295$ ($0 \sim 2^{32} - 1$)
short	정수	2 bytes	$-32,768 \sim 32,767$ ($-2^{15} \sim 2^{15} - 1$)	$0 \sim 65,535$ ($0 \sim 2^{16} - 1$)
char	정수 or 문자	1 byte	$-128 \sim 127$ ($-2^7 \sim 2^7 - 1$)	$0 \sim 255$ ($0 \sim 2^8 - 1$)
float	실수 (부동소수점)	4 bytes	$-3.402823 \times 10^{38} \sim 3.402823 \times 10^{38}$	-
double	실수 (부동소수점)	8 bytes	$-1.797693 \times 10^{308} \sim 1.797693 \times 10^{308}$	-

형식지정자

자료를 어떻게 표현할 것인지에 대해 지정

자료형 : 컴퓨터 정보를 메모리에 어떻게 인식, 표현, 계산하는지 나타냄

형식지정자 : 컴퓨터가 해당 자료/문자를 어떻게 출력/해석하는지 나타냄

형식지정자

%d	10진 정수 (최대 4 bytes)	%u	unsigned 10진 정수 (최대 4 bytes)
%lld	10진 정수 (최대 8 bytes)	%f	실수 (최대 4 bytes)
%o	8진 정수 (최대 4 bytes, 8 bytes의 경우 앞에 11)	%c	문자
%x	16진 정수 (최대 4 bytes, 8 bytes의 경우 앞에 11)	%s	문자열

FORMAT

- %5d 5칸의 공백 확보 후 우측 정렬로 10진 정수 출력
- %.3f 실수. 단, 소수점 3째 자리까지만 출력
- %5.1f 5칸의 공백 확보 후 우측 정렬로 소수점 1째 자리까지 출력
- %p 포인터 변수. 주소 출력

연산자

이미 정의된 연산을 수행하는 문자 또는 기호

단항 연산자(operand가 몇 개인가?) : 단항(a++, (sizeof(a)) 이항(a%b, a>b) 삼항(A > b ? c : d)
(int)a : int형으로 바꿔라 (float)b : float형으로 바꿔라
print("%d",a++) : 출력 후 더해라
print("%d",++a) : 더하고 출력해라

논리연산자

&& AND || OR ! NOT

0이면 거짓, 0이 아닌 값을 참

비트연산자

& AND | OR ~ NOT ^ XOR(다르면 1, 같으면 0)

a=11;//1011 b=3; a<<b : a를 왼쪽비트로 세 칸만큼 움직여라! 결과 101 1000(88)

조건문

상태에 따라 실행해야 하는 동작이 다를 경우

if 조건이 참이면 실행
else 거짓이면 else{} 실행
else if 거짓이면 다른 조건을 비교

```
if (condition1)
    state1;
else
    state2;
state3; >>이렇게 되어 있으면 조건에 상관없이 state3; 실행
```

swtitch case문
break를 써 줘야 함! 아니면 다음 문장까지 실행해버림 >>

```
switch (condition) {
    case A: stateA;
            break;
    case B: stateB;
            break;
    default: stateC;
            break;
}
```

반복문

어떠한 작업을 여러 번 반복하고자 할 때
for(기간; 숫자) vs while(조건; 상태)

for문 실행 순서가 중요!

```
for (①init; condition; ④increment)
{
    ③state; ②
}
```

1. 초기식(initialization)

2. 조건식(condition)

3. 명령문(statement)

4. 증감식(increment)

while문 실행 순서가 중요!

```
while (①condition)
{
    ②state;
}
```

1. 조건식(initialization)

2. 명령문(statement)

do while문 실행 순서, 세미콜론 중요!

```
do ①
{
    ②state;
} while (condition); do while의 끝에는 세미콜론!
```

1. 명령문(statement)

2. 조건식(initialization)

Do while 문 맨 마지막에는 세미콜론이 필요!

continue 다시 조건으로 돌아가라 break 조건문을 탈출해라

goto : 거의 사용되지 않음.

label을 지정해 그 위치로 순간이동.

3주차 - C언어 기초2

배열

일정한 크기의 여러 변수들이 연속된 메모리에 저장되는 구조

`int arr[n]` : 참조가능 범위 0~n-1까지

각 원소는 변수와 동일하게 표현.

초기값을 지정하지 않을 시 dummy value(쓰레기 값)

인덱스를 벗어난 위치를 접근하면 에러 발생 가능

배열의 초기화

0번째 index부터 초기화. 초기화하지 않으면 나머지는 0으로 초기화

배열의 초기값을 모두 0으로 설정하려면 `int arr[10]={0};` 하면 됨

`int arr[]={3,4,5}`

<< 크기를 지정하지 않으면 초기화 한 만큼 자동으로 크기 지정

잘못된 배열의 초기화

`int size=20;`

`int array[size]={10,20,30}` << 배열의 크기를 지정할 때 변수 사용 불가!!!

`int arrat[3]={1,2,3,4}` << 원소의 개수가 배열의 크기보다 크면 에러

다차원배열

`type name[size1][size2] = {val1, val2, val3, val4,... };`

size1 x size2의 행렬이라고 생각하면 편함

초기화 안 한 값들은 0.

`int score[2][3] = {{30,44,67}, {87,43}};`

`int score[2][3] = {{30,44}, {67,87,43}};` << 이렇게 하면 [0][2]는 패스하고 초기화

다차원배열(3차원)

`type name[size1][size2][size3]= {val1, val2, val3, val4,... };`

[0][0][0]부터 [0,0,1].... 순으로 채워짐

초기화 안 한 값들은 0으로 초기화

함수

: 특정 작업을 수행하기 위한 여러 명령어들의 모음

함수 (function)

• 함수의 호출

`type ret = name(type var1, type var2);`

반환 값을 담을 변수

함수의 이름

매개변수

• 함수의 정의

`type name(type var1, type var2) { state; return ret; }`

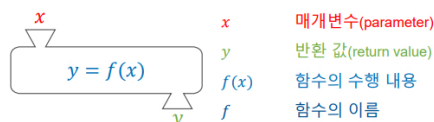
반환 값의 자료형(type)

함수의 이름

매개변수

수행내용

반환 값



함수 관련 용어

- 함수를 사용하는 행위: 함수를 “호출(Call)”한다.
- 함수가 작업 후 반환하는 값: 리턴 값 (Return value)
- 함수가 실행될 때 전달받는 값: 파라미터(Parameter)와 인수(Argument)
 - 파라미터 (Parameter)** : 함수의 정의에서 값을 전달받아 저장하는 변수
 - 인수 (Argument)** : 함수를 호출할 때 전달되는 값

```
#include <stdio.h>                                week3_ex10.c
int main (int argc, char **argv) {
    int i;
    printf("# of parameters : %d\n", argc);
    for (i = 0; i < argc; i++)
        printf("arg(%d):%s\n", i, argv[i]);
    return 0;
}
```

>> test.exe를 입력하고 엔터치기 전에 뒤에 무언가를 더 입력하면 인수로 그 값이 전달된다.

>> argc는 인자의 개수 argv는 각각의 그 값을 배열로 전달받기 위한 포인터 변수

if, test.exe hello world programming 이라고 치고 엔터를 누르면 argc=3이 나오고 for문을 돌려서 argv를 출력하면 첫 번째 파라미터는 실행파일이 위치, 두 번째 파라미터부터 출력됨.

변수의 유효범위(scope)

지역변수 : 함수 안에 있는 변수 - lifetime이 함수가 실행 중일 때만! 이름이 같아도 함수가 다르면 다른 메모리에!

전역변수 : 함수 밖에 있는 변수

static 변수 : 함수가 끝나도 변수가 메모리에서 내려오지 않음! 물론 다른데서 접근은 불가능.

전역 변수 : 프로그램 시작부터 끝까지 메모리에서 내려오지 않음! 어디서든 접근 가능.

호출이 정의보다 먼저 될때?

맨 앞에 뼈대만 선언해놓고 구현은 뒤에다가 놓기 가능!

4주차 - 포인터

포인터

: 메모리의 주소를 참조하기 위한 자료형

메모리의 주소 : 1바이트(8비트) 단위로 이루어진 고유한 숫자

- 변수의 선언 → type *name;
- 메모리의 주소 → & 연산자 사용
- 메모리의 역참조 → * 연산자 사용

```
int main (int argc, char **argv) {  
    int a;  
    int *b;  
  
    a = 10;           //a 변수에 10을 넣기  
    b = &a;           //a의 주소를 포인터 변수인 b에 넣기  
    *b = 20;          //20을 b de-referencing; b가 가리키고 있는 곳에 넣어라  
  
    printf("a :%d\n", a); //a의 값을 출력해라  
    printf("b :%d\n",*b); //b가 가리키고 있는 곳으로 가서 출력해라  
    return 0;  
}
```

포인터의 필요성

* 함수는 자신 내부메모리 밖에 생성된 변수를 모른다.

>> 따라서 외부메모리에 접근하기 위해서는 그 주소를 알아야 한다.

swap()이 그 예시!

포인터 자료형의 크기

포인터에 담기는 값은 “주소”이다!

포인터 자료형의 크기는 항상 동일 - 운영체제에 의존

포인터 변수는 항상 8바이트!!(64비트 윈도우 기준)

Type	Size(bytes)	64bits Windows 기준
int a;	4	
short b;	2	
char c;	1	
double d;	8	
float f;	4	
int *p;	8	
short *q;	8	
char *r;	8	
double *s;	8	
float *t;	8	

32비트 운영체제에서 주소 4바이트 >> 약 4기가의 RAM 활용 가능

64비트 운영체제에서 주소 8바이트 >> 16M 기가의 RAM 활용 가능

포인터 변수의 형변환

메모리에 있는 내용을 1바이트 단위로 보고 싶다면?

>> 1바이트씩 탐색하는 char형 * 포인터를 사용

```
int a = 0x12345678; char * p_char;
```

```
p_1byte=(char*)&a; //a는 int 변수지만, a의 주소를 char로 형변환해서 p_1byte에 저장
```

--->이 (char *)가 형변환

엔디언 ENDIAN

값을 메모리에 어떻게 저장하는가를 정의

• 값을 메모리에 어떻게 저장하는가를 정의

• Big-endian

- 높은 자리수가 앞으로

12	34	56	78
----	----	----	----

0x12345678

• Little-endian

- 낮은 자리수가 앞으로

78	56	34	12
----	----	----	----

>> 빅인디 리틀인디는 하드웨어 종속적

• 최소단위 → 1byte → (메모리의 최소단위)

빅 : 순차적 높은 자리가 앞으로 12/34/56/78 - 비교연산이 빠름!

리틀 : 역순 낮은 자리가 앞으로 78/56/34/12 - 산술연산, 형변환이 빠름

포인터 변수의 연산

포인터 변수에 1을 더하는 것은 다음 주소로 이동하라는 의미.

int * 포인터 변수는 4바이트씩 읽음.

+1 : 다음 값의 메모리 시작 주소. 4바이트 뒤에 있음.

-1 : 이전 칸의 값으로 이동.

week4_ex9.c 참고!

포인터 변수의 초기화

초기화를 하지 않으면? → dummy value

포인터 변수에 담기는 값은 메모리의 주소 → 상수 값은 위험할 수 있음

변수의 주소값 or NULL으로 초기화하는 것이 좋음.

포인터 <-> 배열

일정한 크기의 여러 변수들이 연속된 메모리에 저장되는 구조.

배열의 접근과 포인터의 연산은 유사하게 동작한다!

동적메모리 할당에서 중요하다

```
arr_p == &array[0]; // True
arr_p+1 == &array[1]; // True
*(arr_p) == array[0]; // True
*(arr_p+3) == array[3]; // True
```

배열의 시작(0번) 주소 == 배열 이름!

```
int arr[5];    &arr[0]==arr    <<같은 의미!
```

여기서 "arr"은 값을 변경할 수 는 없다!!

"배열 이름"을 주소로서 참조는 할 수 있지만, 여기에 값을 더하는 등의 연산 등 불가! >> week4_ex13.c 참

고

5주차 - 함수와 포인터

함수의 정의/선언 위치

함수의 정의는 호출보다 먼저 완료해야 함!

- 함수의 **뼈대만** 선언! - 뼈대에는 자료형만 쓰고 변수 이름은 안 써도 됨!
- 그 후 자유로운 위치에 구현!

호출(Call) : 함수를 사용하는 행위

값에 의한 호출(Call by Value)

- 함수의 매개변수가 일반 변수/상수인 경우

주소에 의한 호출(Call by Reference)

- 함수의 매개변수가 주소(포인터)인 경우

week5_ex1.c <- 파라미터가 포인터 변수 일 때,

week5_ex2.c <- 파라미터가 배열일 때는

→ **참조(또는 주소)에 의한 호출** (Call by **reference or address**)

왜 바뀌지...?

포인터를 매개변수로 받으니까
참조에 의한 호출?

```
void sum_to_arr(int *arr, int N) {  
    int i;  
    for (i = 0; i < 5; i++)  
        arr[i] += N;  
}
```

week5_ex1.c

배열 자체를 매개변수로 받으니까
값에 의한 호출?

```
void sum_to_arr(int arr[], int N) {  
    int i;  
    for (i = 0; i < 5; i++)  
        arr[i] += N;  
}
```

week5_ex2.c

포인터 자료형의 크기

포인터에 값기는 값 : "주소" >> 항상 동일하게(8바이트; 64bit 컴) 나옴!

배열에 담기는 값 : sizeof(배열이름)을 하면 (배열의 크기)*(자료형의 크기)

배열을 파라미터로 받을 때!

week5_ex6.c

-> 포인터로 전달받아도 8바이트 주소가 넘어감.

-> 배열 이름으로 전달받아도 8바이트 주소가 넘어감. >> 배열의 요소를 싹다 복사하는 것이 아니다!

>> void func(int arr[]) // int arr[]가 배열의 형태를 띄고 있지만 사실은 포인터 변수였던 것....

다중포인터 (이중포인터)

이중포인터

: "포인터 변수"의 주소 값을 갖는 포인터

int **arr_pp

arr_pp에는 메모리 주소가 담겨 있다.

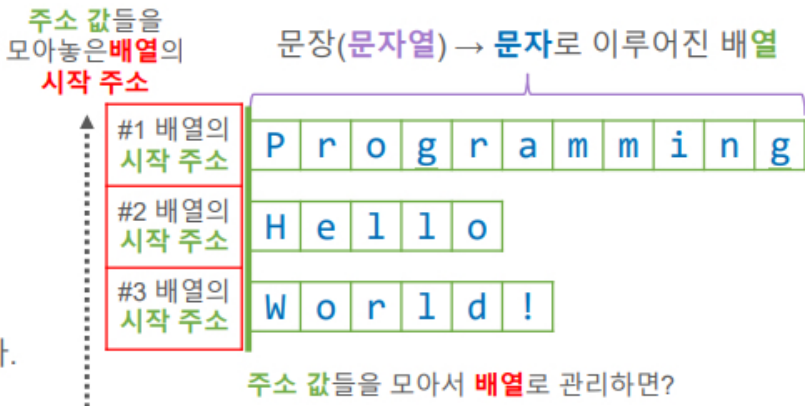
arr_pp를 역참조하면 거기에도 어떤 주소(포인터 값)가 담겨있다.

또다시 역참조를 하면 거기에 int 값이 담겨있다.

사용하는 이유 : N차원 배열의 접근을 위해서는 N중 포인터가 필요하다.

3중은 가끔, 4중 이상은 거의 쓰이지 않음.

Programming Hello World!



해당 값을 저장하기 위한 변수?

`char **argv;`

나는 argv라는 이름을 가진 변수야

나는 주소 값을 갖고 있어

그곳으로 이동하면 또 주소 값이 쓰여 있어

또 그곳으로 이동하면 문자 값이 쓰여 있어

`int score[2][3]` : 3개짜리 배열이 2개있네

2차원 배열에서 배열의 이름은 “첫 번째 원소”의 시작 주소값! week5_ex8.c

`int score[2][3] = {{30, 44, 67}, {87, 43}};`

• 일정한 크기의 메모리

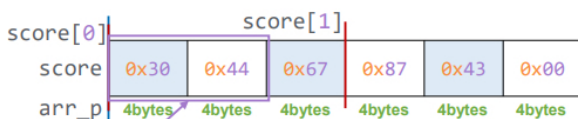
	score[X][0]	score[X][1]	score[X][2]
score[0][X]	30	44	67
score[1][X]	87	43	0



★week5_ex9.c★

이중포인터 변수의 연산

`int score[2][3] = {{0x30, 0x44, 0x67}, {0x87, 0x43}};`
`int **arr_p = score;`



arr_p[0]의 값은? → week5_ex9.c

score[0]의 값은?

유사한 의미
따라서 배열의 이름은?

	score	arr_p
score[0][1]	0x30	*(arr_p + 0)
score[0][1]	0x44	
score[0][2]	0x67	*(arr_p + 1)
score[1][0]	0x87	
score[1][1]	0x43	*(arr_p + 2)
score[1][2]	0x00	

arr_p[0][0]의 값은? → `*(*(arr_p + 0) + 0)` → `*(0x0000004400000030 + 0)`

포인터 변수는 두 번 역참조 불가!!! 더블 포인터여도 애는 그냥 주소만 저장할 뿐... 한번만 역참조 가능

```
arr_p:000000000061FE00
arr_p[0]:30
arr_p[1]:67
score:000000000061FE00
score[0]:000000000061FE00
score[1]:000000000061FE0C
score[0][0]:30
```

wee5 ex14.c

score+1 연산을 하면 score[1][0]의 주소가 나옴...

* 배열 포인터는 배열을 가리키는 단일 포인터로, 이 포인터가 가리키는 배열의 크기와 배열 내 요소의 크기를 고려하여 크기가 다르게 나타납니다.

7주차 - 문자와 문자열

문자 or 1바이트 크기의 정수를 저장하기 위한 변수
초기 C언어는 ASCII 코드를 기준으로 문자를 처리
초기 ASCII Code - 7비트 -> 128개 문자
이후 Extended ASCII Code - 8비트 -> 256개 문자

CR(Carriage Return)-커서 맨앞으로 /r LF(Line Feed)-줄이 올리기 /n

%c 앞에 띄어쓰기 한칸를 하면 공백문자(띄어쓰기, 탭, 줄바꿈)을 무시한다!

문자로 이루어진 배열

```
char str1[] = "Hello World"           -> 12바이트
char str2[] = {'H', 'e', 'l', 'l', 'o', ' ', 'W', 'o'... }; -> 11바이트
```

str2를 %s로 출력하면 출력하고 뒤에 이상한 값이 온다! -> NULL문자가 지정이 안 되어있기 때문
 \0을 해 줘야 한다!

```
char str1 [10] = "Hello";
char str2 [10] = {'H', 'e', 'l', 'l', 'o'};
printf("%s\n", str1);
printf("%s\n", str2);
```

→ 이걸 위 예제랑 달리 출력이 정상적이네?

∴ 문자열의 초기화를 하고 값을 지정 안 한 애들은 **0으로 초기화**되기 때문에!!!

>>> %s에서 문자를 아스키코드로 읽으면 0 == NULL문자라 빈칸으로 출력됨

- int **scanf**(const char * format, ...); 공백문자(띄, 탭, 줄) 등 전까지만 입력
- char* **gets**(char* str); 엔터 전까지 모두입력-> 메모리 초과... 위험
- char* **fgets**(char* str, int count, FILE* stream); 엔터 전|count-1까지 입력받음

char str[13]에 13글자인 "Hello World~!"를 입력		
scanf()는 띄어쓰기 전까지만 입력	- Hello	
gets()는 전체(13글자)를 입력	- Hello World~!	할당받은거 이상으로 메모리 사용... 비정상
fgets()는 12글자+\0까지 입력	- Hello World~	제일 정상이구나!
fgets(str, sizeof(str), stdin);	//Sizeof를 사용하면 매번 입력받지 않아도 됨!	

week7_ex9.c -> 문자열 2개를 연속해서 입력받으려는데... 이상하네?????

Buffer 버퍼

입력을 임시로 저장하는 공간

사용자가 키보드로 입력하면 바로 변수/배열메모리에 할당되지 않음.

-> 엔터를 입력할때까지는 임시공간에 데이터를 저장

문자 하나하나를 복사 vs 모든 입력을 저장 후 한번에 복사

Flush 플러쉬

```
int fflush(FILE *stream); //애를 사이에 넣어주면 버퍼에 남아있던 애들이 없어짐!
```

strlen() 문자열의 길이 알아내기

```
char str[13]="Hello"; //sizeof(str) 하면 13이 나옴. 왜? 메모리 할당된만큼 보여주니까
```

```
size_t strlen(const char* str); // "NULL 문자 만날 때까지" 존재하는 문자의 개수를 반환함.  
//size_t는 그냥 정수형이다... 로만 알아두기
```

week7_ex12.c

```
>>strlen같이 문자열 다루는 함수를 쓰기 위해서는 include <string.h>해야 함!!!
```

문자열 비교

```
str1[13] = "Hello World";
```

```
str2[13] = "Hello World";
```

```
if(str1==str2)
```

```
    printf("Same");
```

```
else
```

```
    printf("different ");
```

```
>> 결과 : different 왜...??? 배열의 이름은 배열의 첫 번째 원소의 주소값이다.
```

strcmp() 문자열 비교함수

week7_ex15.c

```
int strcmp(const char* str1, const char* str2); // str1과 str2를 비교
```

```
int strcmp(const char* str1, const char* str2, size_t n); // str1과 str2를 앞에서부터 n개만큼 비교
```

ASCII 코드 상	앞에것이	앞에있으면 -1	뒤에있으면 1	같으면 0
------------	------	----------	---------	-------

strcpy() 문자열 복사

```
char* strcpy(char * dst, const char * src) // src의 값을 dst에 복사한다! 뒤엣걸 앞으로
```

```
char* strncpy(char * dst, const char * src, size_t n) //n개만큼 src의 값을 dst에 복사한다! 뒤엣걸 앞으로
```

만약 복사하려는 것보다 복사받는 배열의 길이가 짧으면 길이를 넘어가게 되어 위험하다.

> strncpy를 사용하고 n값을 dst보다 작은 값으로 설정하여 메모리를 보호하자

문자열 관련 추가 함수

```
char* strcat(char* dest, const char* origin); //origin문자열의 내용을 dest에 이어붙임 순서대로 붙임
```

```
char* strncat(char* dest, const char* origin, size_t n); //n개만큼 이어붙임
```

```
int* sprintf(char * buffer, const char *format,...); //출력한다. 근데 buffer에
```

```
int* snprintf(char * buffer, size_t_n, const char * format,...); // n개만큼 buffer에 출력
```

strtok() 문자열 분리

```
char *strtok(char * str, const char *delim);
```

str 문자를 delim 문자열에 포함된 문자 기준을 분할해라!

```
char str[] = "Hello Everyone~How are you?Nice to meet you!My name is Mingyu Park";
```

```
char delim[] = "!~?"; //셋중에 하나라도 나오면 동작
```

해당하는 걸 찾으면 그 기호를 \0(NULL 문자)로 바꿈. 그 후 문자열의 시작 주소값을 반환

>>그래서 그 이후를 보려면? 자른 곳의 주소를 알려줘야겠네??

```
token = strtok(token + strlen(token) + 1, delim); <<< 이렇게 해야 한다!! 동영상 1:13:34
```

겁나 복잡하네....

```
token=strtok(NULL, delim) //첫 인자를 NULL로 하면 이전에 자른 문자열을 기억함!
```

그러면 초기에만 반환해주면 됨...

week7_ex21.c

변수? 상수? 배열? 포인터?

week7_ex_22~26 직접 해보기

‘주소’는 ‘값’이다!!! 할당받을 수 없음

‘포인터(형 변수)’는 ‘변수’다! 할당받을 수 있음.

포인터에 의해 초기화된 값(문자열)은 상수이다!

week7_ex_22~26 직접 해보기

/////아래 코드는 불가능!!

```
char str1[] = "Hello"; // 배열을 만들고 문자열을 저장
```

```
printf("Before:%s\n", str1);
```

```
str1 = "World!"; // 배열의 이름은 시작주소 값(상수)이므로 변경 불가능
```

```
printf("After:%s\n", str1);
```

/////아래 코드는 가능!

```
char *str2 = "Hello"; // 포인터"변수"에 문자열(상수들)을 생성 후 주소를 저장
```

```
printf("Before:%s\n", str2);
```

```
str2 = "World!"; // str2는 포인터"변수"이므로 다른 메모리 주소를 가리킬 수 있다.
```

```
printf("After:%s\n", str2);
```

포인터는 등호로 바꿀수 있고 Strcpy 안됨

배열은 등호로 바꿀 수 없고 strcpy 됨